



Gateway API Deployment Strategies

- Gateway API can be used to implement modern deployment strategies such as:
 - Rolling Update
 - Canary Deployment
 - Blue-Green Deployment
 - Traffic Splitting (A/B Testing)



What is Traffic Splitting (A/B Testing) in Gateway API ?

- Traffic splitting allows you to send different percentages of traffic to different backend services (Versions) .
 - Example: 90% Traffic to v1 , 10% Traffic to v2
 - HTTPRoute YAML

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: app-route
spec:
  parentRefs:
  - name: my-gateway
  rules:
  - backendRefs:
    - name: app-v1-service
      port: 80
      weight: 90 # 90% ---> app-v1-service
    - name: app-v2-service
      port: 80
      weight: 10 # 10% ---> app-v2-service
```

- Services
- For Blue-Green and Canary deployments in Gateway API, you typically need 2 Services because each version of the application should have its own Service .

```
apiVersion: v1
kind: Service
metadata:
  name: app-v1-service
```

```
spec:
  selector:
    version: v1
  ports:
  - port: 80
---
apiVersion: v1
kind: Service
metadata:
  name: app-v2-service
spec:
  selector:
    version: v2
  ports:
  - port: 80
```

Gateway API Deployment Strategies

 Deployment Type	 One-Line Answer	 Traffic Example	 Best Use Case
 Traffic Splitting / A/B Testing	Distributes traffic across multiple backend services (versions) using <code>backendRefs.weight</code> .	50% → v1 + 50% → v2	A/B Testing ★★★★
 Blue-Green Deployment	Maintains two environments and switches 100% traffic from old version to new version.	100% Blue → 100% Green	Major Releases ★★★★★
 Canary Deployment	Gradually shifts a small percentage of traffic to a new version before full rollout.	90% → v1 + 10% → v2 → 100% → v2	Safe Production Rollouts ★★★★★

 Feature	 Traffic Splitting	 Blue-Green Deployment	 Canary Deployment
 Downtime	✗ No	✗ No	✗ No
 Risk Level	● Medium	● Low	● Very Low
 Rollback	✓ Easy	✓ Very Easy	✓ Easy
 Gateway	<code>backendRefs.weight</code>	<code>backendRefs.weight</code>	<code>backendRefs.weight</code>

 Feature	 Traffic Splitting	 Blue-Green Deployment	 Canary Deployment
API Feature			
 Example	50% v1, 50% v2	100% Blue → 100% Green	90% v1, 10% v2
 Best Use Case	A/B Testing	Major Releases	Safe Production Rollouts
 User Exposure	Users see different versions simultaneously	Users see only one environment at a time	Small percentage of users see new version first
 Testing Capability	Excellent for experiments	Excellent for full-environment validation	Excellent for production validation

Deployment Traffic Flow Examples

 Deployment Strategy	 Traffic Flow Example	 Purpose	 Key Benefit
 Traffic Splitting (A/B Testing)	100% Traffic → 50% → v1 , 50% → v2	Compare multiple versions simultaneously	User behavior testing
 Blue-Green Deployment	Before: 100% → Blue After: 100% → Green	Switch entire workload to new version	Fast rollback & zero downtime
 Canary Deployment	Phase 1: 90% → v1 , 10% → v2 Phase 2: 70% → v1 , 30% → v2 Phase 3: 100% → v2	Gradually release new version	Lowest deployment risk
 Rolling Update (Default)	Before: 100% → v1 During: Gradually replace v1 Pods with v2 Pods After: 100% → v2	Update application with zero downtime	Default Kubernetes strategy, Replace Pods Gradually

Strategy	Answer
 Rolling Update	Gradually replaces old Pods with new Pods without downtime. v1 Pods → gradually replaced by v2 Pods
 Traffic Splitting	Distributes traffic between multiple versions using weighted routing. Test Both Versions
 Blue-Green	Switches all traffic between two identical environments.
 Canary	Gradually shifts a small percentage of traffic to a new version before full rollout.

Kubernetes Rollout vs Rollback

 Feature	 Rollout	 Rollback
 Purpose	Deploy a new version of an application ("Move Forward")	Revert to a previous stable version ("Move Backward")
 Meaning	Update Pods with a new image/configuration	Undo a failed or problematic deployment
 Direction	Old Version → New Version	New Version → Previous Version
 When Used	Application upgrade	Deployment failure or bug
 Downtime	❌ No (Rolling Update)	❌ No
 Command	kubectl rollout restart deploy/myapp kubectl set image deploy/myapp app=nginx:1.27	kubectl rollout undo deploy/myapp
 History Required	Creates a new revision	Uses previous revision
 Example	v1 → v2	v2 → v1
 Goal	Release new features	Restore service quickly